API Automation Testing compact cheatsheet

STEP 1 Java

1.1 Java · Rest Assured

What API automation is and what to check An API test sends a real HTTP request to a service and checks the response, with no browser involved.

Project setup with Maven Create a Maven project and add the dependencies below. Test code lives in src/test/java , and files such as JSON schemas live in src/test/resources .

Your first test: GET a booking This test asks for booking 1 and checks three things: the status, the content type, and that the first name is a non empty string.

Building requests: params, headers and cookies Real requests carry more than a path. Rest Assured gives a method for each part of the request.

Create, update and delete (CRUD) There are three common ways to send a JSON body: a text block, a Map , or a Java object that Jackson turns into JSON. All three produce the same request.

Assertions with Hamcrest and GPath Rest Assured reads values with GPath , the path language from Groovy, and compares them with Hamcrest matchers. A few patterns cover almost every check.

Extracting values and chaining calls Most real flows need a value from one response in the next request: a token, a new id, a link. Rest Assured gives three ways to pull values out.

Authentication: token cookie, Basic, Bearer, OAuth 2 and API keys APIs protect data in a few standard ways. Restful Booker accepts a token cookie or Basic auth. Most other APIs you test will use a Bearer token, an API key or OAuth 2.

Reusable specifications and filters Copying the same base URI, headers and checks into every test makes a suite hard to change. A RequestSpecification holds shared request settings and a ResponseSpecification holds shared checks.

Data driven tests with DataProvider One test method can run with many rows of data. TestNG feeds each row from a @DataProvider . This is how you cover boundary values and invalid inputs without copying tests.

JSON Schema validation A JSON Schema describes the shape of a response: which fields exist, their types, which are required, and allowed patterns.

POJOs, records and serialization Turning a Java object into JSON is serialization . Turning JSON back into an object is deserialization . Rest Assured does both with Jackson whenever it is on the classpath. Java records make the model classes short.

Designing a real framework Tests that call given() directly are fine for twenty tests. For hundreds, split the code into layers so that a change to an endpoint is fixed in one place. This is the API version of the Page Object pattern.

Mocking other services with WireMock Sometimes the API you test calls another service that is slow, costly or not built yet, such as a payment gateway. WireMock starts a fake HTTP server that returns answers you define.

Parallel runs, reports and CI A mature suite runs on every pull request, in parallel, and publishes a report. TestNG handles parallel runs through testng.xml .

Expert topics: polling, uploads, GraphQL, contracts and more Some APIs process work in the background, so the result appears a few seconds later. Never use Thread.sleep . Poll with a time limit using Awaitility.

Real project essentials: environments, secrets, fake data, safe logging and cleanup In a company, the same Rest Assured suite runs on a laptop, in QA and in staging. Passwords come from the CI secret store, every run makes its own realistic data, failing calls are logged with the token hidden, and ...

Real world scenario: a hotel front desk day A guest calls the front desk. The receptionist books a room, the guest asks for breakfast to be added, then the trip is cancelled.

EXAM In a Selenium or Playwright UI framework, API calls are the fastest way to create test data. Create a booking with one POST, then open the UI and check it shows up. This cuts minutes from every UI test.

TRAP `.auth().basic()` waits for the server to ask for a password with a 401 before it sends one. Restful Booker answers 403 instead, so the password is never sent. Use `.auth().preemptive().basic()`, which sends it on the first request.

TRAP JSON numbers arrive as `Integer`, `Long`, `Float` or `Double` depending on their size. `equalTo(300)` passes for 300 but `equalTo(300L)` fails. For money, check with `closeTo` or extract as `BigDecimal` through a POJO.

EXAM Chained tests should create their own data. Do not depend on booking 1 existing on a shared server, because someone else may delete it. Create, use, then clean up in an `@AfterMethod`.

TRAP Never put real passwords or tokens in code. Read them from environment variables or a secrets store, and keep them out of logs. The blacklist filter in the framework lesson hides them in Rest Assured logs.

TRAP Setting `RestAssured.baseURI` or `RestAssured.requestSpecification` changes a global value. That is fine for a single thread, but in parallel runs each test should pass its own spec with `given().spec(spec)`.

EXAM Restful Booker answers `500` when a required field is missing. A good API should answer `400 BAD REQUEST` with a message. Finding gaps like this is exactly the kind of bug API testers report.

TRAP Logs end up in CI artifacts that many people can read. Always blacklist `Authorization`, `Cookie` and any custom API key header before you turn request logging on.

EXAM The unique guest name makes the search step reliable on a shared server where other people create bookings at the same time.

1.2 Java · Playwright

Why test APIs with Playwright An API test sends an HTTP request and checks the answer, without a browser. Playwright makes this possible with playwright.request().newContext() , which opens no browser at all.

Project setup with Maven and JUnit Run the suite with mvn test , one class with mvn test -Dtest=FirstTest , and point it at another server with mvn test -DbaseUrl=http://localhost:3001 .

Your first test: GET a booking Create one Playwright and one APIRequestContext for the test class, use it in every test, and close both at the end. The context keeps the base URL, default headers and cookies.

Building requests with RequestOptions Every call takes an optional RequestOptions . It holds query parameters, headers, the body, timeouts and more.

Create, update and delete (CRUD) setData accepts a Map , which Playwright turns into JSON and labels with Content-Type: application/json . A String is sent as it is, so add the header yourself.

Assertions and reading JSON Playwright gives the body as text or bytes. Parse it with Jackson into a JsonNode and walk it with JSON Pointer paths like /bookingdates/checkin . JUnit assertAll runs every check and reports all failures together.

Chaining calls and authentication Log in, keep the token, and send it with later calls. Restful Booker wants the token in a cookie. Most other APIs want a Bearer token in the Authorization header.

Data driven tests with @ParameterizedTest JUnit runs one method once per row of data. Use @CsvSource for small tables, @MethodSource for objects, and @CsvFileSource for a file.

JSON Schema validation A schema describes the shape of a response. The networknt validator checks a response against it and lists every problem it finds.

Framework design: a client class and a base test Move HTTP details into a client class with one method per endpoint, and move setup into a base class. Tests then only describe behaviour.

Mixing UI and API in one test This is where Playwright shines for QA. Each browser context has its own request() that shares cookies with the pages in it. Three patterns save the most time.

Parallel runs, thread safety and CI JUnit can run test classes in parallel. Playwright objects are not thread safe , so every thread needs its own Playwright and context.

Real project essentials: environments, secrets, test data, logging and cleanup A real Playwright for Java suite runs in several places, must not print secrets, creates unique data so parallel CI jobs never clash, says exactly what went wrong when it fails, and removes what it created.

Real world scenario: a hotel front desk day The receptionist logs in, books a room, adds breakfast, and later cancels. The test follows the same steps and includes the check a hacker would try.

EXAM If your team runs Playwright for UI, this is the natural first step into API testing. Every UI test can start with API calls that create exactly the data it needs.

TRAP Restful Booker answers 403 instead of 401 when auth is missing. Playwright's `setHttpCredentials` waits for a 401 before sending the password, unless you add `.setSend(HttpCredentialsSend.ALWAYS)`. Sending the `Authorization` header yourself, as above, always works.

EXAM Boundary and invalid data rows are where most API bugs hide: empty strings, very long strings, negative prices, a checkout before the checkin, and dates in the wrong format.

EXAM Mocking with `page.route` lets you test how the UI handles slow answers, empty lists and server errors, which are hard to produce on a real server.

1.3 Java · HttpClient

Why learn the native way An API test sends an HTTP request and checks the response. Libraries make this shorter, but each one sits on top of an HTTP client like this one.

Setup: from one file to a Maven project Java can run a single source file directly, with no build tool. This is the fastest way to try the client. The script below calls the API and exits with code 1 if a check fails, so CI treats it as a failure.

Building requests: query strings, headers, methods HttpClient does not build query strings for you. Encode each value with URLEncoder , or names with spaces and symbols will break the URL.

Create, update and delete (CRUD) Bodies are sent with a BodyPublisher . Text blocks keep JSON readable. Remember to set Content-Type , because the client does not guess it.

Reading JSON and asserting well Parse the body once into a JsonNode with Jackson. Read nested values with JSON Pointer paths. Turn the body into a record when you want to compare whole objects.

Authentication and cookies A CookieManager on the client stores cookies the server sets with Set-Cookie and sends them back on later calls, like a browser. You can also add one yourself.

Async calls and parallel requests sendAsync returns at once with a CompletableFuture . Start many calls together and wait for all of them. This is how you check that an API behaves under a little concurrency.

A stub server with the JDK alone The JDK includes a small HTTP server in com.sun.net.httpserver . Use it as a stub for a service your code depends on, or to test your own client code against answers you control, including errors and slow responses.

Schema checks and a small framework Wrap the raw client in a small class so tests stay short. Add schema validation with networknt, which reads the schema and the body as plain strings.

A quick load smoke with virtual threads This is not a replacement for a load testing tool, but a quick check that an endpoint stays fast under a small burst. Send 50 calls at once, collect the times, and check the 95th percentile.

Real project essentials: environments, secrets, logging with redaction and cleanup Even with no libraries, a native suite can behave like a professional one. The JDK already has everything needed: system properties and environment variables for config, System.Logger for logs, UUID for unique data, and ...

Real world scenario: a release gate health check Before a release goes live, the team wants a quick yes or no: is the booking service healthy and does a full booking journey still work? The check must run anywhere a JDK exists, finish in seconds, and print a clear ...

EXAM Many companies have internal tools that check service health before a release. A single Java file using HttpClient can do that with nothing to install except the JDK.

TRAP Some header names are restricted and cannot be set, such as `Host`, `Connection` and `Content-Length`. The client sets them itself.

TRAP The JDK `Authenticator` only answers a 401 challenge. Restful Booker answers 403, so build the `Authorization` header yourself as shown.

EXAM A table of path and expected status is a quick way to cover routing, missing ids and wrong id types in one place. Add rows whenever a bug is found so it never returns.

TRAP Never run load against a shared public API such as Restful Booker beyond a small burst. Use your own test environment for real load testing with tools such as Gatling, k6 or JMeter.

TRAP The regular expressions here keep the example free of libraries. In a real suite, parse JSON with Jackson as the schema lesson does. Regex parsing breaks as soon as the response format changes.

STEP 2 Python

2.1 Python · requests and pytest

What API automation is and what to check An API test sends a real HTTP request and checks the response, with no browser. API tests are fast and stable, so teams keep many of them in the middle of the test pyramid.

Fixtures and conftest.py A fixture is a function that prepares something a test needs. A test asks for it by naming it as an argument. Code after yield runs after the test, which makes cleanup automatic.

Schema checks with jsonschema and pydantic Two good options. jsonschema checks a response against a JSON Schema file, which can be shared with other teams and tools.

Framework design: a client class Keep URLs and HTTP details in one client class. Tests call methods with clear names and only contain checks. When an endpoint changes, you fix it in one place.

Mocking HTTP with responses and respx When you test code that calls an API, such as your own client class, you often want to control the answers. responses intercepts calls made with requests. respx does the same for httpx.

Async tests with httpx httpx has the same style as requests, plus an async client. With asyncio.gather you can send many calls at once. This is useful for quick concurrency checks and for testing async services.

Retries, polling and flaky networks Real networks drop calls now and then. Retry the safe calls at the HTTP layer so a single blip does not fail the build, and poll properly when an API finishes work in the background.

Expert topics: property based testing, contracts and load Schemathesis reads an OpenAPI spec and generates thousands of requests, including odd values you would not think of. It checks that the API never answers 500 and that every response matches the spec.

Real project essentials: environments, secrets, fake data, logging and cleanup Tutorials talk to one server with one password. Real teams run the same suite against local, QA and staging, keep passwords out of Git, need fresh realistic data for every run, and want to see the exact request and ...

Real world scenario: a hotel front desk day A receptionist books a room for a guest, adds breakfast later, and then the guest cancels. Someone without a login also tries to cancel the booking. The test proves each step, using fixtures for login.

EXAM pytest prints the actual values when an `assert` fails, for example `assert 404 == 200`. You do not need special assertion methods, which keeps tests easy to read for the whole team.

EXAM Restful Booker returns 200 with `{"reason": "Bad credentials"}` for a wrong password. Many teams would log this as a bug, because 401 is the correct status. Your test documents the real behaviour and makes it visible.

TRAP Keep secrets in environment variables or a CI secret store. Never print tokens in test output.

EXAM Mocks test your code, not the real API. Use them for unit tests of helpers and clients, and keep separate tests that call the real service.

TRAP Retries hide real problems if overused. Log every retry, keep the count low, and never retry a 4xx, since the request itself is wrong.

2.2 Python · Playwright

Why test APIs with Playwright for Python An API test sends an HTTP request and checks the response, with no browser. Playwright only starts a browser when a test asks for a page , so pure API tests run as fast as tests written with requests.

Assertions, readable failures and polling Playwright for Python has one web first assertion for API responses, expect(res).to_be_ok() . Everything else is a plain pytest assert , which prints both sides when it fails.

Fixtures for tokens and test data Fixtures prepare what a test needs and clean it up afterwards. The code after yield is the cleanup, and pytest runs it even when the test fails. A session scoped fixture runs once per run, which suits a login token.

Framework design: a client class and fixtures As a suite grows, keep URLs and headers in one class with one method per endpoint. Tests then read like business steps, and a changed endpoint is fixed in one place.

UI and API together This is where Playwright beats every other Python HTTP client. Inside a UI test, page.request shares cookies with the browser, page.expect_response captures what the page sends, and page.route fakes API answers.

Async API calls for speed The sync API sends one call at a time. The async API in playwright.async_api can keep many calls in flight with asyncio.gather . Use it for seeding lots of data, quick load checks and concurrency tests.

Real project essentials: environments, secrets, fake data, logging and cleanup A company suite runs against several servers, keeps passwords out of Git, creates fresh data for every run, explains its own failures, and leaves the shared QA database tidy.

Real world scenario: a hotel front desk day The receptionist logs in, books a room, adds breakfast, and the guest later cancels. Someone without a login also tries to cancel. Each step is a small function, so a failure names the exact step that broke.

EXAM If your team already writes Playwright UI tests in Python, add API tests to the same repo. Start by replacing slow UI setup steps, such as creating a booking through five screens, with a single API call.

TRAP In Python, `status`, `ok`, `headers` and `url` are properties, while `json()`, `text()` and `body()` are methods. Writing `res.status()` raises a TypeError.

TRAP One long test like this stops at the first failed step and leaves the booking behind. The fixtures lesson shows how to give every test its own booking that is always cleaned up.

TRAP Without `"send": "always"`, `http_credentials` waits for a 401 challenge before sending the password. Restful Booker answers 403, so the password would never be sent and the test would fail.

EXAM For every protected endpoint, test at least three cases: no credentials, wrong credentials and a valid token that lacks the right role. Then read the data again to prove nothing changed.

EXAM `expect_response` lets a UI test assert on the exact request the page sent. It catches bugs where the screen looks right but the wrong data reaches the server.

TRAP Do not share one sync Playwright object between threads. For parallel work use the async API, or run tests in separate processes with pytest-xdist.

TRAP Faker data is random, so a failure might not repeat. Print the data in the log, or seed Faker with `Faker.seed(1234)` in a run you need to reproduce.

EXAM Real front desk systems add more steps, such as payment and email. The pattern stays the same: create your own data, act through the API, check the state after every step, and clean up.

STEP 3 JavaScript

3.1 JavaScript · Postman and Newman

From manual requests to automated tests An API test sends a request and checks the response. In Postman you already send requests by hand.

Your first request with a test Create a request GET {{baseUrl}}/ping . Open Scripts , then Post-response , and add the script below. Press Send, then open Test Results under the response.

Variables and their scopes Write {{name}} anywhere in a URL, header or body and Postman replaces it before sending. In scripts you read and write variables with pm .

CRUD requests and chaining with variables Each request saves what the next one needs. The login stores token , the create stores bookingId , and later requests use {{token}} and {{bookingId}} .

Assertions cookbook pm.expect is the Chai assertion library. Chains read like sentences and give clear failure messages.

Authorization tab and auth scripts Set auth once on the collection or a folder, then choose Inherit auth from parent on each request. Postman supports the common types directly.

Data driven runs with CSV or JSON Run the same requests once per row of a data file. Column names become variables, so {{firstname}} in the body is filled from the row. In scripts, read the row with pm.iterationData .

JSON Schema checks pm.response.to.have.jsonSchema(schema) validates the body with Ajv, which is built into the Postman sandbox. One schema check replaces many field checks and catches renamed or retyped fields.

Controlling the run: setNextRequest, skips and loops By default the runner plays requests from top to bottom. Scripts can change that.

Newman: reporters, options and running from Node Newman is also a Node library, so you can run collections from a script and decide what to do with the results.

Real project essentials: environments, secrets, safe logging and cleanup When a whole team shares one workspace, a few habits keep it safe: one environment per server, secrets that never sync to the cloud or Git, a guard that stops destructive tests on production, logs that never print ...

Real world scenario: the front desk collection A receptionist logs in, books a room, the guest adds breakfast, someone without a login tries to cancel, and finally the booking is cancelled. Here is the full collection, built from the scripts in the earlier lessons.

EXAM This is the friendliest path from manual to automated API testing. You keep the requests you already trust and add checks one at a time.

TRAP Mark secrets such as passwords as SECRET type in the environment, and do not commit exported environments that contain real secrets.

EXAM Data files are a simple way to hand test design to the whole team. Testers add rows for boundary values and special characters without touching any script.

TRAP Jumps only work in the Collection Runner, Newman and the Postman CLI, not when you press Send on a single request. Always add a limit to loops.

TRAP The INITIAL VALUE of a variable syncs to the Postman cloud and is shared with everyone in the workspace. Put real secrets only in the CURRENT VALUE , in Vault, or in CI secrets.

EXAM Import this file into Postman with IMPORT , then run it in the Collection Runner to see every test result in the app.

3.2 JavaScript · Playwright

Why test APIs with Playwright Test An API test sends a request and checks the response, with no browser. Playwright Test does not start a browser unless a test asks for a page , so pure API tests run in milliseconds and in parallel across workers.

Custom fixtures for tokens and test data Fixtures prepare what a test needs and clean it up afterwards. Code after await use(value) is the cleanup, and it runs even when the test fails. A worker fixture runs once per worker process, which suits a login token.

Data driven tests Playwright has no special data annotation. A normal loop that calls test() creates one named test per row, and each row can run in parallel.

UI and API together Inside a UI test, page.request shares cookies with the browser. Three patterns save QA teams the most time.

Real project essentials: environments, secrets, test data, logging and cleanup A company suite runs against several servers, keeps secrets out of Git, makes unique data so parallel workers never clash, attaches the exact exchange to the report when something fails, and leaves the shared QA ...

Real world scenario: a hotel front desk day The receptionist logs in, books a room, adds breakfast, and the guest later cancels. Someone without a login also tries to cancel. Each step is a test.step , so the report reads like the story.

EXAM If your UI tests already use Playwright, API tests can live in the same repo and CI job. Start by replacing slow UI setup steps with API calls.

TRAP `test.describe.serial` runs the tests in order and stops after the first failure. Use it only for true step by step flows. Independent tests with fixtures, shown next, are better in most cases.

TRAP Without `send: 'always'`, `httpCredentials` waits for a 401 challenge. Restful Booker answers 403, so the password would never be sent.

EXAM `waitForResponse` lets a UI test assert on the exact request the page sent. This catches bugs where the screen looks right but the wrong data reaches the server.

TRAP Attachments and traces include request headers. If your API uses an `Authorization` header, avoid attaching raw headers, or replace the token with *** before you attach.

TRAP Inside Playwright Test, do not call `api.tracing.start()` yourself when `trace` is set in the config. The runner already records every request context the test creates, and a second start fails with `Tracing has been already started`. Manual tracing is for scripts and other languages that do not use the runner.

3.3 JavaScript · fetch

Why go native An API test sends a request and checks the response. With Node 22 or newer you can do this with no packages at all, which means no supply chain risk, no version upgrades and instant installs in CI.

Assertions with node:assert For older Node versions, a tiny helper gives the same partial match.

Hooks, shared login and auth checks before runs once before the tests in its block, beforeEach before every test, and the after versions clean up. Use them to log in once and to create and delete test data.

Mocking fetch and time When you test your own code that calls an API, replace fetch with a mock. The test controls every answer and checks every call, with no network.

Concurrency and a quick latency check Sort n times from fastest to slowest. The p95 sits at

Framework design with the client The BookingClient from the mocking lesson keeps tests short. The same class is tested with mocks and then used for real calls.

Real project essentials: environments, secrets, fake data, logging and cleanup Node.js now ships most of what a professional API suite needs: .env loading, random ids, test hooks for cleanup, and a built in way to watch every fetch call without wrapping it. Here is how a real team uses them.

Real world scenario: a hotel front desk day The receptionist logs in, books a room, adds breakfast, and the guest cancels. Someone without a login also tries to cancel. Using t.test subtests, each step appears as its own line in the report.

EXAM fetch works the same in Node and in the browser console. Testers can try a request in DevTools, then paste the same lines into a test file.

TRAP Forgetting `Content-Type: application/json` is the most common fetch mistake. Many servers then ignore the body or answer 400 or 500.

TRAP Node's fetch does not keep cookies between calls like a browser does. Send the `Cookie` header yourself, or read `res.headers.getSetCookie()` and store the values.

TRAP Snapshot the SHAPE or stable fields, not whole bodies with ids and timestamps. A snapshot that changes on every run trains the team to update it without reading it.

STEP 4 TypeScript

4.1 TypeScript · Playwright

Why write API tests in TypeScript Imagine a team of ten testers sharing one test suite for a payments API. The backend renames amount to amountPaise .

Typed models: interfaces, utility types and satisfies Put the shapes of your API in one file. Every test imports them, so a change to the API is a change in one place.

Run time checks with Zod: one schema, one type A type tells the editor what you expect. A Zod schema checks what actually arrived. Better still, Zod can produce the type from the schema, so you write the shape once and get both.

A typed client class with generics A client class hides URLs and headers, so tests read like the business story. With types and Zod, every method returns data that is both checked and typed.

Typed fixtures for clients and test data test.extend takes two type parameters: fixtures created for each test, and fixtures created once per worker. The editor then knows exactly what each test can ask for.

Types generated from the OpenAPI contract Hand written types can drift away from the real API. In most companies the backend publishes an OpenAPI file, and the test project generates its types from it.

Framework design: environments, logging and a type gate Real projects run the same tests against local, QA and staging, keep secrets out of the code, and print useful details when something fails. A small typed config file covers all three.

Real world scenario: a hotel front desk day The receptionist logs in, books a room, adds breakfast, and the guest later cancels. Someone without a login also tries to cancel.

TRAP Types exist only while you write and check code. They are removed before the code runs, so they cannot check what the server really sent. That is why this track also uses ZOD , which checks response bodies at run time.

EXAM Most new Playwright projects in companies are started in TypeScript. If you can read and write typed tests, you can join those teams on day one.

TRAP openapi-typescript 7.13 still lists TypeScript 5 as its peer dependency, so installing it next to TypeScript 7 fails with a peer dependency error. Running it with `npx openapi-typescript@7.13.0` keeps it out of your project and avoids the clash. Checking peer dependencies like this is normal work when a big new compiler version arrives.

EXAM Add the generate step to CI and fail the build if `src/generated/booker.ts` changed but was not committed. That way nobody can change the API contract without the tests noticing.